2-HOUR HANDS-ON WORKSHOP

# Workshop on Building a Mini Vending Machine Integrated with QRIS Payments

Live demonstration · Python application · Arduino Uno · QRIS sandbox


Presented by

## Ajang Rahmat, S.Kom

Electrical Engineering · Nusa Putra University

**nutral.id**


Build with the supplied source code. Learn by observing, wiring, uploading, and testing.

# Watch the complete experience first

The instructor demonstrates the goal before explaining the parts.

1. **Select a product**
2. **Show the QRIS checkout**
3. **Confirm the test payment**
4. **Light the matching LED**

**NOTICE**

The QR payment shown in the workshop must use instructor-approved sandbox credentials.

**ASK**

What did the application do? What did the Arduino do? Where did the QR payment fit?

# Answer the big questions before we build

Start with the purpose and user experience—not with code.

### WHAT?

A desktop vending interface connected to an Arduino-controlled LED demo.

### WHY?

To make software, payment status, serial messages, and physical output tangible.

### WHO?

A customer selects a product; an operator configures and monitors the machine.

### WHERE?

Python runs on a computer; Arduino Uno runs the hardware sketch.

### WHEN?

The output is requested only after the payment flow reports success.

### HOW?

Python sends a serial command; Arduino maps the requested slot to an LED output.

# Four parts work together

The workshop uses LEDs as a safe, visible stand-in for vending hardware.

**CUSTOMER**

**Selects a product and scans a QR**

→

**PYTHON APP**

**Shows products, cart, and payment**

→

**SERIAL LINK**

**Carries commands through USB**

→

**ARDUINO**

**Turns on the LED for the chosen slot**

**IMPORTANT DISTINCTION**

The application, payment provider, serial connection, and output circuit are separate layers. A success message is not proof that a physical product was dispensed.

# Use the right tool for each responsibility

A small prototype is easier to understand when each part has one clear job.

| | |
|---|---|
| **Python** | Fast to read and change; coordinates the interface, cart, and payment flow. |
| **Tkinter** | Included with standard Python; creates the desktop window and buttons. |
| **Arduino Uno** | Reads a small sketch and controls hardware outputs predictably. |
| **USB serial** | Simple text messages connect the computer to the microcontroller. |
| **QRIS sandbox** | Allows a controlled payment-flow demonstration without production payments. |

# A demonstration, then a guided build

Installers and hardware should be ready before the hands-on session begins.

| | |
|---|---|
| 00–08 | Instructor live demonstration |
| 08–20 | Project purpose and 5W1H |
| 20–40 | Install tools, create a virtual environment, install packages |
| 40–55 | Run and explore the supplied Python application |
| 55–65 | Electronics, pin map, and safety |
| 65–80 | Wire one LED; extend to three or six |
| 80–100 | Write, upload, and test the Arduino sketch |
| 100–110 | Test the serial protocol and integration path |
| 110–115 | Review and evaluation |
| 115–120 | Questions and answers |

# Prepare computers and kits

Pre-downloading installers saves hands-on time; installation and verification happen together.

### SOFTWARE

Python 3.11.9 (Windows installer)
Visual Studio Code
Arduino IDE
Project source folder

### HARDWARE PER PARTICIPANT / TEAM

Arduino Uno and USB data cable
Breadboard and jumper wires
1–6 LEDs
One 220–330 Ω resistor per LED

### INSTRUCTOR PREPARATION

Share the project source and installers in advance. Prepare a verified sandbox account for the opening demo; participants must not use production payment credentials.

# Install Python 3.11.9, VS Code, and Arduino IDE

Use official download sources; choose the Windows versions that match the classroom computers.

1. **Install Python 3.11.9. Enable the Python Launcher if offered.**
2. **Install Visual Studio Code and open the supplied project folder.**
3. **Install Arduino IDE and the Arduino AVR Boards / Uno support.**
4. **Connect the Uno with a USB data cable; note its COM port.**

```
py -3.11 --version
```

**Expected: Python 3.11.9**

# Create a project-local virtual environment

Run commands in the VS Code terminal with the supplied project folder as the current directory.

```
py -3.11 -m venv .venv
.venv\Scripts\python.exe -m pip install --upgrade pip
.venv\Scripts\python.exe -m pip install -r requirements.txt
.venv\Scripts\python.exe -c "import tkinter, serial, PIL, qrcode; print('Setup OK')"
.venv\Scripts\python.exe main.py
```

**WHY A VIRTUAL ENVIRONMENT?**

It keeps this project's packages separate from other Python projects.

**IF SOMETHING FAILS**

Check the current folder, Python version, and full error message before retrying.

# Read the project map before changing anything

Participants use the finished Python source; this workshop is not a from-scratch Python build.

| | |
|---|---|
| `main.py` | Connects the application flow and button actions. |
| `vending_ui.py` | Builds the Tkinter interface. |
| `inventory.py · cart.py` | Define products, stock, and cart rules. |
| `configuration.py` | Selects demo, sandbox, or production settings. |
| `serial_controller.py` | Sends commands and checks Arduino responses. |
| `firmware/.../vending_controller.ino` | Arduino sketch for the vending controller. |

**Goal: recognize responsibilities and follow one event across files.**

# Start with a working baseline

Do not edit the payment configuration or copy secrets into the workshop folder.

```
.venv\Scripts\python.exe main.py
```

### CHECK THE APP

Window opens
Product cards and stock are visible
Select a product and observe the cart

### PAYMENT DEMO

No provider key: local simulation only. With an approved sandbox key: sandbox QRIS checkout. Never scan or test production credentials in class.

### SAFE PRACTICE

Keep MODE_DEMO=1 and SUMOPOD_SANDBOX=1. Do not display .env or share API keys.

# Start with a low-voltage LED circuit

An LED is a simple output indicator; it is not a motor or a vending mechanism.

### ONE LED NEEDS

Arduino Uno
Breadboard
LED

### SAFETY RULES

Disconnect USB before rewiring. Use one resistor per LED. Never connect an LED directly to a pin. Do not connect mains voltage, motors, or relay loads in this activity.

**Long LED leg = anode (+) Short leg / flat edge = cathode (−)**

### UNO PIN BUDGET

Each output pin drives only its own LED through a resistor. Stay within Arduino Uno pin-current limits; use the instructor-approved circuit and do not power external loads from GPIO pins.

# Build and inspect one complete circuit

Keep the Arduino disconnected from USB until the wiring has been checked.

```
Arduino D2 ── 220–330 Ω resistor ── LED anode (+)
Arduino GND ──────────────────────────── LED cathode (–)
```

**CHECK THE PATH**

D2 → resistor → long LED leg → short LED leg → GND

**BEFORE POWER**

Check polarity, resistor placement, and that no rail is shorted.

**When the one-LED test works, duplicate the circuit on D3–D7.**

# One product slot maps to one output pin

Start with one LED. Add more only after the first output works.

**Slot 1**

D2 → resistor → LED → GND

**Slot 2**

D3 → resistor → LED → GND

**Slot 3**

D4 → resistor → LED → GND

**Slot 4**

D5 → resistor → LED → GND

**Slot 5**

D6 → resistor → LED → GND

**Slot 6**

D7 → resistor → LED → GND

**WIRING CHALLENGE**

Each LED has its own resistor and its own output pin. Keep every cathode on the common GND rail.

# Use the same serial protocol as the Python app

This classroom sketch drives LEDs directly with active-HIGH outputs; it is not the relay firmware.

```
const byte LEDS[] = {2, 3, 4, 5, 6, 7};
const byte SLOT_COUNT = sizeof(LEDS) / sizeof(LEDS[0]);

void setup() {
  Serial.begin(9600);
  Serial.setTimeout(100);
  for (byte i = 0; i < SLOT_COUNT; i++) {
    pinMode(LEDS[i], OUTPUT);
    digitalWrite(LEDS[i], LOW);
  }
  Serial.println("READY");
}

void loop() {
  if (!Serial.available()) return;
  String command = Serial.readStringUntil('\n');
  command.trim();
  if (command == "PING") { Serial.println("PONG"); return; }
  if (!command.startsWith("DISPENSE,")) {
    Serial.println("ERROR,UNKNOWN_COMMAND"); return;
  }
  int slot = command.substring(9).toInt();
  if (slot < 1 || slot > SLOT_COUNT) {
```

# Compile first, then upload to the Uno

Use the USB cable, correct board, correct port, and the serial speed configured in the sketch.

1. **Open the sketch in Arduino IDE.**
2. **Tools → Board → Arduino Uno; Tools → Port → the Uno COM port.**
3. **Click Verify. Resolve compile errors before uploading.**
4. **Click Upload; wait for the upload-complete message.**
5. **Open Serial Monitor at 9600 baud; select Newline as line ending.**

**Close Serial Monitor before Python opens the same COM port.**

# Test the board before connecting the desktop app

Send each command in Serial Monitor and compare the expected response.

| | | |
|---|---|---|
| **Board startup** | `READY` | Printed after reset |
| **Send: PING** | `PONG` | Board is responsive |
| **Send: DISPENSE,1** | `DONE,1` | D2 LED flashes once |
| **Send: DISPENSE,3** | `DONE,3` | D4 LED flashes once, if wired |
| **Send: DISPENSE,9** | `ERROR,INVALID_SLOT` | Invalid slot is rejected |

Every command ends with a newline. Both sides must use 9600 baud.

# The supplied serial module checks every response

Find these functions in serial_controller.py; do not rewrite them during the workshop.

```
serial_controller.connect("COM3", 9600)
serial_controller.check_ready()       # Python sends PING; expects PONG
serial_controller.dispense(slot)      # sends DISPENSE,<slot>
                                       # waits for DONE,<slot>
serial_controller.disconnect()
```

**PORT**

Replace COM3 with the port shown by Device Manager / Arduino IDE.

**PORT OWNERSHIP**

Close Arduino Serial Monitor before launching the Python app.

# Demonstrate QRIS sandbox to LED output

Only the instructor should run this step with approved sandbox credentials.

| | |
|---|---|
| `MODE_DEMO=0` | Enables the application's dispenser path. |
| `SUMOPOD_SANDBOX=1` | Keeps the payment endpoint in sandbox. |
| `Arduino connected` | The app must receive READY and answer PING with PONG. |
| `Sandbox payment confirmed` | The selected slot LED flashes after the payment is confirmed. |

**STOP SAFELY**

Never change SUMOPOD_SANDBOX to 0 during this workshop. Do not share API keys. Sandbox payments can still create test transactions and must use

# Check one layer at a time

Read the exact error and verify the nearest connection before changing code.

| | |
|---|---|
| **Python command not found** | Check Python 3.11.9 installation and the py launcher. |
| **Package import error** | Use the .venv Python executable and install requirements.txt. |
| **Upload / COM error** | Check USB data cable, board selection, port selection, and driver. |
| **No READY / PONG** | Close Serial Monitor; verify 9600 baud, newline, and the uploaded sketch. |
| **Wrong LED lights** | Compare the slot-to-pin map and check LED polarity / wiring. |

# Explain the system back in your own words

Use the diagram and your successful test as evidence of understanding.

### PYTHON APPLICATION

Where are the products, cart rules, UI, payment configuration, and serial calls?

### ARDUINO SKETCH

What do setup(), loop(), pinMode(), digitalWrite(), and Serial do?

### COMMUNICATION

What does PING/PONG verify? Why does DISPENSE include a slot number?

### PAYMENT AND SAFETY

When may the output happen? Why must sandbox and production credentials stay separate?

**Success criteria: app runs · circuit is safe · sketch uploads · serial test passes**

# What worked, and what would you improve?

Take one minute to reflect, then bring your questions.

**REFLECT**

Which part was easiest? Which part failed first? How did you isolate the problem?

**NEXT CHALLENGE**

Add three LEDs, then six. Map each product slot to one LED and confirm the serial response.

# Questions and answers

Ajang Rahmat, S.Kom · Electrical Engineering, Nusa Putra University · nutral.id